LEDGER

# Building with Ledger




OLIVIER FREYSSINET · SOFTWARE ENGINEER, LEDGER

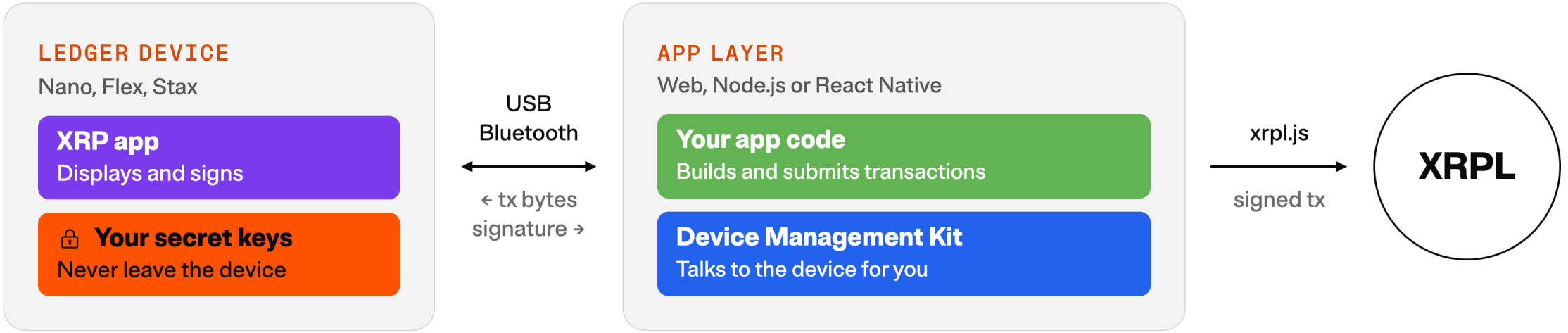
LEDGER

01 · BUILD ON TOP OF LEDGER

# The tools you can use

The Device Management Kit, its XRP signer, and Speculos when you have no device

# What lives where



# Device Management Kit




- **Device discovery**
- **Connection**
- **Data exchange**
- **State management**
- **Transaction signing**
- **App management**

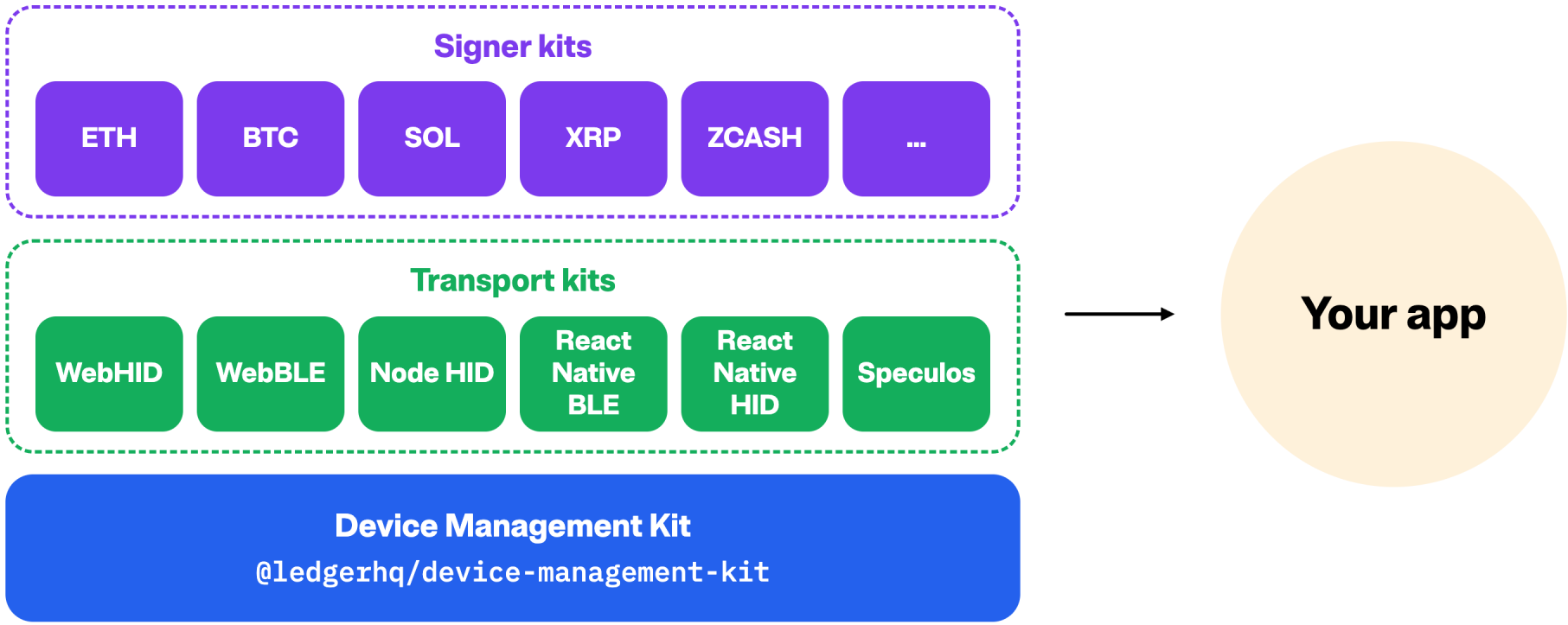
**Try it** app.devicesdk.ledger.com

**npm** `@ledgerhq/device-management-kit`

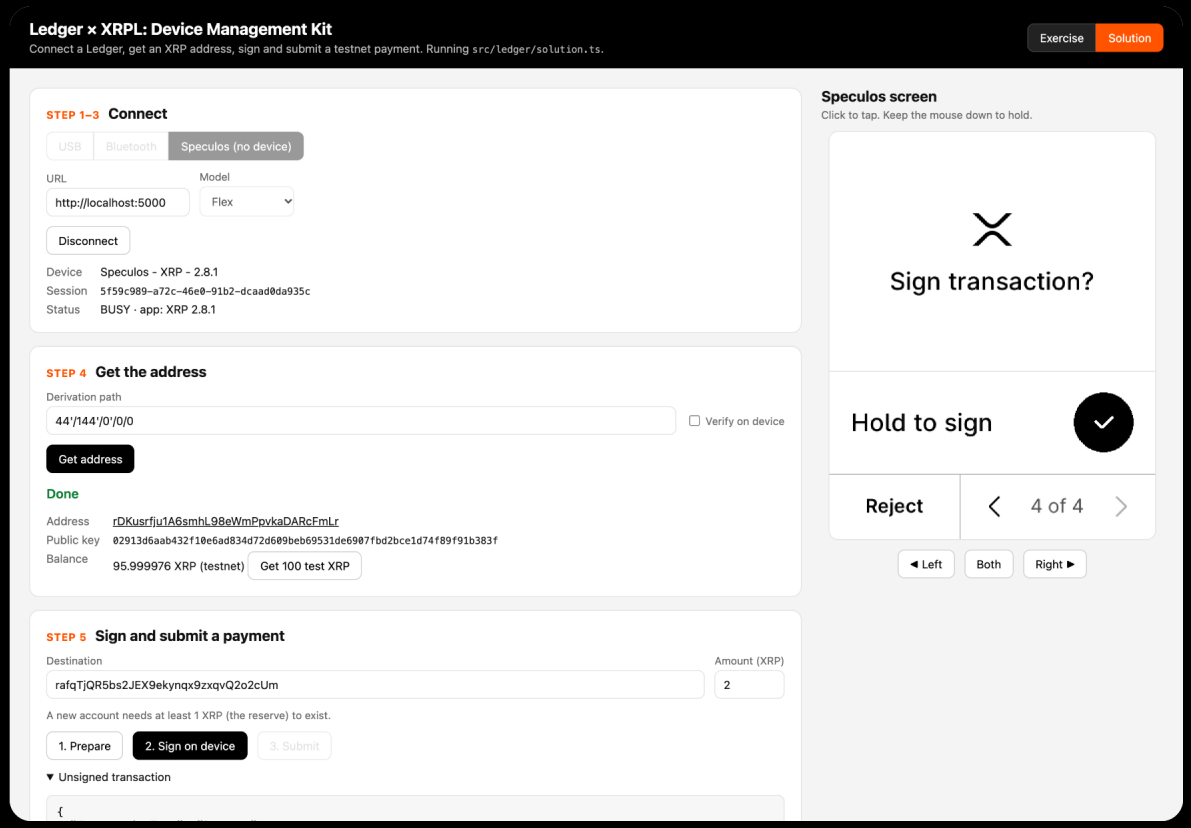
# From LedgerJS to the DMK

| | LedgerJS · hw-app-xrp LEGACY | DMK · XRP signer kit USE THIS |
|---|---|---|
| **Abstraction** | Low level: raw APDUs, status words | Device actions run the whole flow |
| **Device state** | Errors to catch, poll and retry | Live states: unlock, open app, sign |
| **Opening the app** | Up to you or the user | Done for you |
| **Bluetooth** | Legacy transports, unstable | Web BLE and React Native BLE |
| **New features** | Maintenance only, no new features | All new features, for XRP and others |

# A modular approach



# Live demo: connect, sign, submit




1. Connect over USB, Bluetooth or Speculos
2. Get the address and 100 test XRP
3. Review and sign on the device
4. Submit to the XRPL testnet

github.com/LedgerHQ/device-management-kit-ts-workshop

# Build the Device Management Kit 1/5

```
import { DeviceManagementKitBuilder } from "@ledgerhq/device-management-kit";
import { webHidTransportFactory } from "@ledgerhq/device-transport-kit-web-hid"; // + web-ble, speculos

const dmk = new DeviceManagementKitBuilder()
  .addTransport(webHidTransportFactory)        // USB
  .addTransport(webBleTransportFactory)        // Bluetooth
  .addTransport(speculosTransportFactory(url)) // Speculos emulator
  .build();
```

One instance for the whole app. On Node.js, add nodeHidTransportFactory instead.

# Discover and connect 2/5

```
import { firstValueFrom } from "rxjs";
import { webHidIdentifier } from "@ledgerhq/device-transport-kit-web-hid";

const [discoveredDevice] = await firstValueFrom(
  dmk.startDiscovering({ transport: webHidIdentifier }),
);

const sessionId = await dmk.connect({ discoveredDevice });
```

# Create the XRP signer 3/5

```
import { SignerXrpBuilder } from "@ledgerhq/device-signer-kit-xrp";

const signerXrp = new SignerXrpBuilder({ dmk, sessionId }).build();
```

Three methods: getAppConfig, getAddress, signTransaction. Each one opens the XRP app on the device first.

# Get the XRP address 4/5

```
import { DeviceActionStatus } from "@ledgerhq/device-management-kit";

signerXrp.getAddress("44'/144'/0'/0/0", { checkOnDevice: true })
  .observable.subscribe((state) => {
    switch (state.status) {
      case DeviceActionStatus.Pending: // "unlock-device", "verify-address"…
        return showPrompt(state.intermediateValue.requiredUserInteraction);
      case DeviceActionStatus.Completed: // { address: "r…", publicKey }
        return setAccount(state.output);
      case DeviceActionStatus.Error:
        return showError(state.error);
    }
  });
```

# Sign an XRPL transaction 5/5

```
import { DeviceActionStatus, hexaStringToBuffer } from "@ledgerhq/device-management-kit";

signerXrp.signTransaction("44'/144'/0'/0/0", hexaStringToBuffer(unsignedTx)!)
  .observable.subscribe((state) => {
    switch (state.status) {
      case DeviceActionStatus.Pending: // "sign-transaction": review on the device
        return showPrompt(state.intermediateValue.requiredUserInteraction);
      case DeviceActionStatus.Completed: // DER signature, Uint8Array
        return addSignatureAndSubmit(state.output);
      case DeviceActionStatus.Error:
        return showError(state.error);
    }
  });
```

# Submit to the XRPL with xrpl.js

```
import { Client, encode, xrpToDrops } from "xrpl";
import { bufferToHexaString } from "@ledgerhq/device-management-kit";

const client = new Client("wss://s.altnet.rippletest.net:51233"); await client.connect();

const tx = await client.autofill({
  TransactionType: "Payment", Account: address, Destination: "r…", Amount: xrpToDrops("2"),
  SigningPubKey: publicKey.toUpperCase(), // the device's key, in uppercase
});
const unsignedTx = encode(tx);              // NOT encodeForSigning()
// … signerXrp.signTransaction(path, bytes) → signature
tx.TxnSignature = bufferToHexaString(signature, false).toUpperCase();
await client.submitAndWait(encode(tx));
```

# Before you start

GOT A DEVICE

## Ledger Wallet

Install the XRP app on your device, from Ledger Wallet.

ledger.com/ledger-wallet

NO DEVICE

## Speculos

Run the real XRP app on your laptop, with Docker. Our workshop repo has it ready to run. Next slide.

`xrpl-hackathon/speculos`

BUILDING AN AGENT

## Node.js

Agents, bots and backends: the same signer code, with the Node HID transport.

`device-transport-kit-node-hid`

# Build on Ledger without a device

If you don't have a Ledger device, Speculos emulates a Ledger running the real XRP app, so you can develop without one.

```
$ docker pull ghcr.io/ledgerhq/speculos:latest
$ cd xrpl-hackathon/speculos
$ ./run-speculos.sh stax
```

```
import { DeviceManagementKitBuilder, DeviceModelId } from "@ledgerhq/device-management-kit";
import { speculosTransportFactory } from "@ledgerhq/device-transport-kit-speculos";

const dmk = new DeviceManagementKitBuilder()
  .addTransport(speculosTransportFactory("http://127.0.0.1:5000", false, DeviceModelId.STAX))
  .build();
```

⚠ Never use a real seed or real funds on Speculos. Its seed sits in plain text on your machine: use test seeds and testnet only.

LEDGER

02 · LEDGER BOUNTIES

# $10,000

in Ledger bounties, across three tracks.

# Pick your track

TRACK 01 · CORE

**$4,000**

## Protocol Innovation

Bring the modern Ledger stack into XRPL, and lean into the gaps. A rough prototype that fills a real gap beats a polished rebuild.

- Put the new device-signer-kit-xrp to work: integrate it in a real app, find where it breaks, extend it. Bug reports and PRs count.
- Human-readable previews for TrustSet, OfferCreate, AMM or Escrow, before you sign.
- A Ledger-secured DEX, AMM or trust-line dApp for the flows Ledger Wallet can't do today.

TRACK 02

**$3,000**

## Agentic Finance

Ledger as the trust layer for agents: device-backed security at the center.

- Agents that pay for APIs and services, with x402-style pay-per-use.
- Human in the loop: the Ledger approves high-risk actions before funds move.
- Agents that settle on XRPL: per-call micropayments or escrow-backed work, approved on the device.

TRACK 03

**$3,000**

## Why Not

The open track: anything, as long as it settles at least one transaction on XRPL or the EVM sidechain.

- A polished consumer app or wallet experience that signs with a Ledger.
- Tooling that makes XRPL easier for people: address books, spending monitors, onboarding.
- Something fun: an on-chain game, a collectible, an interactive demo.

# What we like

The same bar applies to all three tracks.

- Ledger used as a real trust layer, not just wallet branding.
- Signs on a Ledger device or on Speculos, against mainnet, testnet or the EVM sidechain.
- Fills a real gap rather than rebuilding something that already exists.
- A working demo we can run, not a slide deck.
- Clear boundaries between what is automated and what a human approves.
- Reusable by other developers where it makes sense: a library beats a one-off.

# Documentation feedback

We judge the Developer Experience feedback as much as the code. Help us improve our docs.

## What to include

- ✓ Feedback on overall experience using Ledger docs & SDKs
- ✓ Identify gaps, confusing flows, or missing context
- ✓ Specific improvements with screenshots or PRs

## Bonus points

- \+ Better tutorial or code sample ideas
- \+ Navigation or search improvements for the portal
- \+ "Time-saver" suggestions for future integrators

LEDGER

FIND US ALL WEEKEND

# Come build with us

| | |
|---|---|
| **Akram El Milligy** | DevRel / Product Manager |
| **Etienne Waldron** | DevRel / Product Manager |
| **Eva Mirza** | Product Manager |
| **Olivier Freyssinet** | Software Engineer |

**Tracks, workshop code and slides**

developers.ledger.com/**xrplnyc**

# What the XRP app signs today

SHOWN ON THE DEVICE

## Clear signing

Payment (XRP and IOU tokens), TrustSet, Offers, Escrow, Checks, Payment channels, AccountSet, SignerListSet, Tickets, NFToken mint, burn, create and cancel offers, AMM, Clawback

OFF BY DEFAULT

## Needs Blind signing

Credentials, Batch, DelegateSet, PermissionedDomains, DID, NFTokenModify, NFTokenAcceptOffer, AMMClawback

Also Payments and Offers that use DomainID, CredentialIDs or Delegate. Turn it on in the XRP app's settings.

REJECTED

## Can't be signed

MPT tokens: payments in MPT and most MPT transactions. OracleSet. XChain bridges.

Also: at most 8 memos or signers per transaction, and no message signing.